

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A

MINOR PROJECT REPORT

ON

EDUMENTX: LOCATION BASED TUTOR DISCOVERY PLATFORM

SUBMITTED BY:

APOORVA PAUDEL (NCE080BCT006)

ASIM POUDEL (NCE080BCT008)

DIBYA SHAKYA (NCE080BCT013)

SUHAN KHADKA (NCE080BCT043)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

August, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled '**EdumentX: Location Based Tutor Discovery Platform**' submitted by **Apoorva Paudel**, **Asim Poudel**, **Dibya Shakya**, and **Suhan Khadka** in partial fulfillment of the requirements for the Bachelor's degree in Computer Engineering.

.....
Supervisor

Er. Sahil Subedi
Senior Lecturer
Department of Electronics and Computer Engineering,
National College of Engineering, IOE, TU.

.....
External examiner

Er. Tika Raj Shrestha
Lecturer

Date of approval: .....

Copyright

The authors have agreed that the Library, Department of Electronics and Computer Engineering, National College of Engineering, and Institute of Engineering may make this report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisor who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the authors of this report and the Department of Electronics and Computer Engineering, National College of Engineering, IOE for any use of the material of this project report. Copying, publication, or other use of this report for financial gain without the approval of the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering, and the authors' written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
National College of Engineering
Talchhikhel, Lalitpur, Nepal.

Acknowledgments

We would like to express our sincere gratitude and highest regards to our project supervisor, **Er. Sahil Subedi**, Senior Lecturer, Department of Electronics and Computer Engineering, for his continuous guidance, valuable suggestions, and constructive feedback throughout this project. His regular reviews, technical suggestions, and practical insights helped us define our project scope clearly and complete the work systematically.

We express our gratitude to the Head of the Department of Electronics and Computer Engineering, **National College of Engineering**, and all the faculty members who provided us with the necessary academic foundation, lab resources, and encouragement during our study.

Finally, we acknowledge the open-source software community and researchers whose prior work in mobile application frameworks, geospatial indexing algorithms, vector databases, and retrieval-augmented generation has provided the foundational tools upon which this project is built.

Abstract

Finding a qualified and trustworthy home tutor in the Kathmandu Valley is mostly an informal and unorganized process. Parents and students usually depend on word-of-mouth recommendations, social media groups, or agency bureaus that charge high broker fees. These traditional methods lack credential verification, distance-based search, and structured scheduling. Existing educational platforms in Nepal mostly focus on online video classes or general classifieds, leaving a clear gap for localized home tutor discovery.

EdumentX is a mobile application developed using React Native and Expo SDK 54 to help students and parents find verified home tutors in their nearby areas. The platform provides an interactive map interface using Google Maps with pin clustering, proximity ranking based on the Haversine distance formula, and integrates a third-party AI-powered Retrieval-Augmented Generation (RAG) API chatbot that helps users find tutors using natural language questions. To ensure safety and credibility, tutors go through an identity verification process where citizenship documents and profile photos are checked by leveraging external optical character recognition (OCR) and face validation APIs before approval. The application supports monthly enrollment requests with weekly time-slot scheduling, group tuition batches, in-app direct messaging, ratings and reviews, and integrates the third-party eSewa digital payment sandbox for a Pro subscription tier. The backend relies on external cloud platforms including Firebase Authentication, Cloud Firestore, and Supabase Edge Functions operating within free tier quotas. The system was tested through 64 automated unit tests, 21 design-token lint tests, and seven Firestore security rule test suites, demonstrating a practical and cost-effective mobile platform for home tutor discovery in Kathmandu Valley.

Keywords: *Location-Based Services, Tutor Discovery, React Native, Firebase Firestore, Haversine Distance, Retrieval-Augmented Generation, eSewa Payment.*

Contents

Page of Approval ii

Copyright iii

Acknowledgements iv

Abstract v

List of Figures ix

List of Tables x

List of Abbreviations xi

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives 3

1.4 Scope 3

2 Literature Review 5

2.1 Related Work 5

2.2 Related Theory 6

2.2.1 Two-Sided Marketplaces and Platform Matching 6

2.2.2 Core Data Entities and Relational Modeling 6

2.2.3 Location-Based Services and Haversine Distance Formula 8

2.2.4 Geofencing and Radius-Based Filtering 8

2.2.5 Nearest-Neighbor Ranking by Geographic Distance 8

2.2.6 Retrieval-Augmented Generation (RAG) and Semantic Search 9

2.2.7 Mobile Application Framework (React Native and Expo) 9

2.2.8 Multi-Axis Rating Aggregation and Anti-Tampering 9

3 Methodology 10

3.1 System Architecture and Implementation Framework 10

3.1.1 Mobile Client Architecture . . . 10
3.1.2 Backend Services and Serverless Compute . . . 10
3.2 Module Implementation . . . 11
3.2.1 Authentication and Onboarding Module . . . 11
3.2.2 AI Tutor Verification Module . . . 11
3.2.3 Tutor Discovery and Map Module . . . 12
3.2.4 Enrollment Request Workflow Module . . . 13
3.2.5 Group Batches Management Module . . . 13
3.2.6 In-App Direct Messaging Module . . . 13
3.2.7 Pro Tier and eSewa Payment Module . . . 13
3.2.8 Ratings and Reviews Module . . . 15
3.2.9 AI Chatbot Module . . . 15
3.2.10 Admin Dashboard Module . . . 15

4 System Design **16**
4.1 Requirement Specification . . . 16
4.1.1 Functional Requirements . . . 16
4.1.2 Non-Functional Requirements . . . 18
4.2 System Architecture . . . 18
4.3 Data Flow Diagrams . . . 20
4.3.1 Level 0 Context Diagram . . . 20
4.3.2 Level 1 Data Flow Diagram . . . 20
4.4 Use Case Diagram . . . 21
4.5 AI RAG Chatbot Workflow . . . 23
4.6 Database Schema . . . 24

5 Results and Discussion **25**
5.1 Software Development Approach . . . 25
5.2 Testing, Evaluation and Validation . . . 25
5.2.1 Test Environment Specification . . . 25
5.2.2 Unit Test Cases . . . 26
5.2.3 Integration Test Cases . . . 27
5.2.4 System Journey Test Cases . . . 29
5.2.5 AI Identity Verification and Vision OCR Evaluation . . . 30
5.2.6 Security Test Cases . . . 31
5.2.7 Test Traceability Summary . . . 32

6 Conclusions **33**

7 Limitations and Future Enhancement **35**

7.1 Limitations 35

7.2 Future Enhancement 35

References **37**

Appendices **39**

Appendix: Application Screenshots 39

List of Figures

3.1 Authentication and Onboarding Workflow 11
3.2 Architecture of Tutor Verification Process 12
3.3 Enrollment Request Status Lifecycle 13
3.4 eSewa Payment Transaction Flow 14

4.1 System Architecture of EdumentX 19
4.2 Level 0 Data Flow Diagram (Context Diagram) 20
4.3 Level 1 Data Flow Diagram 21
4.4 Use Case Diagram of EdumentX 22
4.5 RAG-Based AI Chatbot Architecture 23

7.1 EdumentX Mobile Application Screenshots: (a) Onboarding, (b) Authentication, and (c) Student Map Discovery. 39

List of Tables

2.1 Comparison of Related Systems vs. EdumentX 6
2.2 Core Data Entities for EdumentX 7

4.1 Primary Firestore Document Collections and Schema 24

5.1 Test Environment Specifications 25
5.2 Unit Test Case UT-01 – Asia/Kathmandu Timezone Conversion 26
5.3 Unit Test Case UT-02 – SlotKey Syntax Validation 26
5.4 Unit Test Case UT-03 – Haversine Great-Circle Distance Calculation 26
5.5 Unit Test Case UT-04 – Dynamic Today's Sessions Derivation 27
5.6 Unit Test Case UT-05 – SeatsRing SVG Capacity Math 27
5.7 Integration Test Case IT-01 – Signup, Email Verification and Role Routing . 27
5.8 Integration Test Case IT-02 – Tutor Verification Upload and AI Screening . 28
5.9 Integration Test Case IT-03 – Map Pin Clustering and Selection 28
5.10 Integration Test Case IT-04 – Enrollment Request to Roster Transaction . . 28
5.11 Integration Test Case IT-05 – eSewa Payment and Pro Activation 29
5.12 System Journey Test Case SYS-J1 – Student Discovery to Enrollment 29
5.13 System Journey Test Case SYS-J2 – Tutor Onboarding and Schedule Setup . 29
5.14 System Journey Test Case SYS-J3 – Pro Subscription Upgrade 30
5.15 AI Identity Verification – Observed Sandbox Behaviour 30
5.16 Security Test Case ST-01 – Student Location Privacy Protection 31
5.17 Security Test Case ST-02 – Cross-User Profile Modification Guard 31
5.18 Security Test Case ST-03 – Free Tier Capacity Cap Enforcement 31
5.19 Security Test Case ST-04 – Chat Thread Access Authorization 32
5.20 Test Suite Traceability Matrix 32

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
FCM	Firebase Cloud Messaging
GPS	Global Positioning System
HMAC	Hash-based Message Authentication Code
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
IEEE	Institute of Electrical and Electronics Engineers
IOE	Institute of Engineering
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
LBS	Location-Based Service
LLM	Large Language Model
NoSQL	Non-Relational Database
NPR	Nepalese Rupee
OCR	Optical Character Recognition
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RLS	Row Level Security
SDK	Software Development Kit
SEE	Secondary Education Examination
SHA	Secure Hash Algorithm
UI / UX	User Interface / User Experience
WGS84	World Geodetic System 1984

1. Introduction

1.1 Background

Education is one of the most important foundations for personal growth and social progress. Around the world, families invest heavily in their children's education, recognizing that formal schooling alone may not always meet every student's learning needs. When students struggle with certain subjects or need dedicated preparation for important board examinations, many families turn to private tutoring as a way to fill academic gaps [1]. This practice of hiring tutors outside of regular school hours has become widespread globally, as parents seek additional academic support to help their children perform better [2].

Private tutoring, commonly referred to as shadow education, operates as a parallel academic system alongside formal schooling [3]. This supplementary education has grown particularly fast in developing countries, where mainstream school resources vary widely and student-to-teacher ratios remain high. In Nepal, supplementary tuition plays a major role in student preparation, particularly for national board assessments such as the Secondary Education Examination (SEE) and Grade 11/12 (+2) examinations under the National Examination Board. Thapa's study on private and public school competition in Nepal highlights how supplementary tutoring has become an established practice, with approximately 68 percent of Grade 10 students receiving private tutoring to reinforce their classroom learning [4].

At the same time, Nepal has experienced rapid urbanization over the past decade. According to the National Population and Housing Census 2021, the overall literacy rate reached 76.3 percent among the population aged five years and above, while the urban population share increased substantially to 66.17 percent [5]. In urban centers like the Kathmandu Valley, dual-income households have become common. Working parents face strict time limitations in supervising their children's daily coursework and actively search for trusted, qualified home tutors who can deliver focused one-on-one instruction directly at their residence.

Despite this growing demand, Nepal's tutoring market remains fragmented, informal, and unorganized. Parents and students typically rely on word-of-mouth recommendations from friends and relatives, local notice boards, or unstructured social media groups to find tutors. These informal methods lack credential verification, distance-based filtering, and structured scheduling workflows. Furthermore, intermediary tuition agencies charge heavy upfront

commissions from independent tutors (often 50% or more of the first month's payment) while providing zero accountability for ongoing session delivery or tutor safety.

With smartphone penetration in Nepal exceeding 70 percent and internet connectivity expanding across urban centers, the digital infrastructure exists to address these problems through a dedicated mobile application. EdumentX is developed to solve these systemic inefficiencies by providing a transparent, location-first home tutor discovery ecosystem for the Kathmandu Valley. The platform enables students and parents to find nearby verified tutors through an interactive map, enroll in structured monthly plans, and receive personalized recommendations through an AI-powered RAG chatbot.

1.2 Problem Statement

The process of finding a home tutor in Nepal involves several practical difficulties for both students and tutors:

- Lack of Verification and Trust: Parents cannot easily check whether a tutor is qualified, verified, or trustworthy before inviting them into their home. Most families rely on informal contacts, which limits their choices and provides no assurance of teaching quality or safety.

- No Proximity-Based Search: Home tutoring requires physical travel. Existing classifieds and social media posts list only general area names without actual distance or map location, leading to frequent cancellations when tutors realize the commute is impractical.

- Unorganized Communication and Scheduling: Inquiries are usually handled through unstructured phone calls or messaging chats, with no organized records of time slots, subject requirements, or request statuses.

- High Agency Brokerage Fees: Tuition bureaus charge high upfront commissions from independent tutors (such as university students and part-time educators) while offering no persistent platform for reputation building or direct student-tutor relationships.

EdumentX addresses these problems by providing an interactive map with clustered tutor pins, an AI-assisted tutor verification system, a natural language RAG chatbot, slot-based weekly scheduling, and transparent multi-axis reviews.

1.3 Objectives

The main objectives of this project are:

- To develop a location-based mobile application that allows students and parents to discover nearby home tutors in the Kathmandu Valley using interactive map search and a natural language AI chatbot.
- To implement a secure tutor verification system alongside transparent review mechanisms, helping parents make safe and informed hiring decisions.
- To provide a complete management system that streamlines monthly enrollments, slot-based scheduling, group tuition batches, and direct in-app communication between tutors and students.

1.4 Scope

The scope of the EdumentX project includes:

- Target Users: Students, parents, independent home tutors, and platform administrators.
- Geographic Focus: Kathmandu Valley (Kathmandu, Lalitpur, and Bhaktapur), within standard Nepal coordinates (26.35°N to 30.45°N, 80.06°E to 88.20°E).
- Service Type: Physical in-person home tuition matching; online video classes are not included.
- Academic Levels: School and college students covering Grade 1–10, SEE, +2 Science/Management, and undergraduate levels.
- Mobile Platform: Android mobile application built using React Native with Expo SDK 54.
- Discovery Methods: Interactive map search with proximity ranking and conversational AI chatbot.
- Enrollment Model: Monthly enrollment plans (1, 3, 6, or 12 months) with weekly time-slot selection ($day : slot$).
- Verification: Blue Tick badge awarded after document verification (citizenship and photo review).
- Group Learning: Group tuition batches (2 to 6 seats) created by tutors for shared, cost-effective learning.

- Subscription: Pro Tutor tier with eSewa sandbox payment for increased student capacity and visibility.

2. Literature Review

2.1 Related Work

Recent academic literature and commercial systems have explored digital tutoring and local services. This section reviews relevant systems and identifies the specific gaps that EdumentX aims to address.

A 2022 study presented at ASCILITE introduced MENTOR, an intelligent mobile peer tutoring platform designed for higher education institutions [6]. MENTOR matched peer tutors with students based on subject strengths and academic availability. While it demonstrated the value of mobile platforms for tutoring coordination, it did not include any location-based features such as map visualization or proximity ranking. The matching algorithm operated solely on academic criteria without considering the physical commute between participants.

Sharma et al. (2026) described a Local Tutor Finder Application built using Flutter and Firebase [7]. The application integrated Google Maps for proximity-based tutor search and calculated straight-line distances using the Haversine formula. Students could filter tutors by subject, rate, and city. However, the system lacked automated document verification and intelligent recommendation features, requiring students to browse listings manually. The authors noted that AI-based recommendations were left as future scope.

Shari et al. (2025) proposed iTeachU, an AI-driven tutor matching platform utilizing geolocation integration within the Melaka region of Malaysia [8]. The platform combined content-based filtering with the K-Nearest Neighbors (KNN) algorithm to rank tutors by subject preferences and physical distance. It was built using Flutter for the frontend and Flask for the backend. While effective, the user interface was rigid, relying strictly on fixed dropdown menus without conversational natural language support.

In Nepal, online rental platforms such as KothaBhada, GharBheti, and BasoBaas have shown the effectiveness of digital discovery for real estate. However, these systems are built specifically for rental properties. For home tutoring, users still rely on unvetted Facebook groups, WhatsApp chats, or agency bureaus that offer no schedule management, verified ratings, or map visualization. Table 2.1 presents a structured comparison of these systems against EdumentX.

Table 2.1: Comparison of Related Systems vs. EdumentX

Feature	**MENTOR [6]**	**Local Tutor [7]**	**iTeachU [8]**	**EdumentX**
Focus Area	Peer Tutoring	Local Tutors (India)	University Tutors	Home Tutors (Kathmandu)
Platform	Mobile Web	Flutter Android	Flutter / Flask	React Native / Expo
Map Search	None	Basic Pins	Radius Filter	Clustered Avatar Pins
Distance Algorithm	None	Haversine	Haversine + KNN	Haversine + Clustering + Pro Tiering
Conversational AI	None	None	None	RAG Chatbot (Groq API)
Identity Verification	Manual	None	Manual	AI OCR + Face Validation
Slot Scheduling	Basic	None	Time Slots	7x6 Matrix ($day : slot$)
Group Batches	None	None	None	Supported (2–6 Seats)
Payment Model	None	None	Cash	eSewa Pro Subscription
Location Privacy	College Internal	Open Database	Basic RLS	Protected Location Markers

2.2 Related Theory

2.2.1 Two-Sided Marketplaces and Platform Matching

A two-sided marketplace connects two distinct user groups: suppliers (tutors offering educational services) and consumers (students and parents seeking academic instruction) [9]. The value of the platform depends on cross-side network effects: students benefit from a wide selection of verified nearby tutors, while tutors benefit from a consistent stream of localized inquiries without intermediary exploitation.

2.2.2 Core Data Entities and Relational Modeling

The core EdumentX entities are structured across document collections and relational data schemas. Primary keys and document identifiers uniquely identify records, while foreign keys and user identifiers enforce relationships and data integrity. Normalized schemas reduce redundancy and support reliable updates.

Table 2.2: Core Data Entities for EdumentX

Entity	**Primary Key**	**Important Attributes / Foreign Keys**
UserProfile	uid	email, role (student/tutor/admin), status, created_at
StudentProfile	default	uid (FK), full_name, phone, grade, subjects, location_label, saved_tutors
TutorProfile	default	uid (FK), full_name, headline, bio, monthly_rate_npr, student_capacity, enrolled_count, rating, review_count, is_available
LocationAccess	tutor_uid	student_uid (FK), enrollment_id (FK), status, granted_at
TutorListing	uid	full_name, headline, monthly_rate_npr, latitude, longitude, is_available_for_new_students
TutorVerification	uid	document_type, document_url, face_url, ocr_name, match_confidence, status, reviewed_at
EnrollmentRequest	id	tutor_uid (FK), student_uid (FK), subjects, plan_duration, schedule_preference, status
Roster	id	tutor_uid (FK), student_uid (FK), enrollment_id (FK), slot_key, start_date, end_date, status
GroupBatch	batch_id	tutor_uid (FK), name, subject, schedule_slots, max_seats, current_members, monthly_rate_npr
Conversation	id	participant_a (FK), participant_b (FK), last_message, unread_count
Message	id	conversation_id (FK), sender_uid (FK), text, status (sent/read), created_at
Review	id	tutor_uid (FK), student_uid (FK), score, category_ratings, comment, created_at
SubscriptionTransaction	id	tutor_uid (FK), product_id, amount_npr, esewa_ref_id, status, created_at

2.2.3 Location-Based Services and Haversine Distance Formula

Location-Based Services (LBS) deliver context-aware information using a device's real-time geographic position obtained through GPS or network triangulation. In EdumentX, tutor distances are calculated using the Haversine great-circle formula [10], which calculates the shortest surface distance over the spherical Earth ($R \approx 6{,}371$ km):

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{2.1}$$

$$c = 2 \cdot \operatorname{atan2}\left(\sqrt{a},\, \sqrt{1-a}\right), \qquad d = R \cdot c \tag{2.2}$$

where ϕ_1, ϕ_2 are latitudes in radians, λ_1, λ_2 are longitudes in radians, $\Delta\phi = \phi_2 - \phi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$, and d is the calculated distance in kilometers. For the 1 to 20 km distances typical in the Kathmandu Valley, this formula provides accurate proximity calculation.

2.2.4 Geofencing and Radius-Based Filtering

Before executing the trigonometric Haversine formula, EdumentX applies a rectangular bounding box pre-filter to narrow down candidate tutors and optimize mobile device performance [11]. For a given search radius r and student position (ϕ_s, λ_s):

$$\Delta\phi_{\max} = \frac{r}{R} \cdot \frac{180}{\pi}, \qquad \Delta\lambda_{\max} = \frac{r}{R \cdot \cos(\phi_s)} \cdot \frac{180}{\pi} \tag{2.3}$$

Only tutors whose coordinates fall inside this bounding box proceed to the exact Haversine distance check, keeping query execution fast.

2.2.5 Nearest-Neighbor Ranking by Geographic Distance

The tutor recommendation problem is treated as a one-dimensional nearest-neighbor search over geographic distance, following the classical K-nearest neighbors formulation of Cover and Hart [12]. Candidate tutors inside the geofence bounding box (Section 2.2.4) are ranked by their exact Haversine distance from the student's location, in ascending order, so the closest tutors appear first:

$$R = \operatorname{sort}_d\left(\{t_1, t_2, \ldots, t_k\}\right), \quad d(t_i) = d_{\text{haversine}}(s, t_i) \tag{2.4}$$

On top of this ordering, the platform applies a stable tier promotion: Pro-subscribed tutors are moved to the top of the list while preserving the original distance order within each tier. This keeps the ranking transparent and explainable so that a student always sees why a tutor appears where they do while still giving subscribers the visibility they pay for.

2.2.6 Retrieval-Augmented Generation (RAG) and Semantic Search

Retrieval-Augmented Generation (RAG) is an architecture that combines information retrieval with large language model generation to produce responses grounded in private, real-time data rather than static model weights [13]. In EdumentX, RAG parses student natural language queries into structured parameters (subject, grade, budget, locality), fetches matching active tutor profiles from Firestore, and injects them as factual context into the Groq LLM prompt. This approach prevents model hallucinations and ensures recommendations reflect live platform listings [14].

2.2.7 Mobile Application Framework (React Native and Expo)

React Native enables cross-platform mobile development from a unified TypeScript codebase. Expo SDK 54 provides a managed native toolchain with pre-configured modules for device location (`expo-location`), native camera and image pickers (`expo-image-picker`), file system storage, and native Google Maps rendering (`expo-maps`).

2.2.8 Multi-Axis Rating Aggregation and Anti-Tampering

To prevent rating bias and unilateral manipulation, EdumentX breaks tutor ratings into five qualitative dimensions: Communication, Subject Knowledge, Punctuality, Clarity, and Engagement. When an enrolled student submits a review, a Firestore transaction recalculates running averages atomically:

$$R_{\text{new}} = \frac{R_{\text{old}} \cdot N_{\text{old}} + S_{\text{new}}}{N_{\text{old}} + 1} \tag{2.5}$$

3. Methodology

This chapter details the problem-solving methodology implemented in EdumentX, covering the system development framework, technical architecture, and module-level implementation across the application lifecycle.

3.1 System Architecture and Implementation Framework

3.1.1 Mobile Client Architecture

The mobile application is developed in React Native (v0.81.5) using Expo SDK 54:

- Design System: NativeWind 4.2 provides universal styling through Tailwind CSS classes mapped to native primitives without runtime styling overhead. Custom ESLint rules enforce design token compliance across all components.
- File-Based Routing: Expo Router v6 manages flat stack navigation under `src/app/`, featuring layout guards in `_layout.tsx` that protect private routes based on authentication and verification state.
- Fluid Animations: React Native Reanimated and `expo-haptics` deliver 100ms spring animations for bottom sheets, modal drawers, and interactive buttons.
- Client-Side State: Zustand stores manage session state, user roles, and recent conversation history.

3.1.2 Backend Services and Serverless Compute

The backend architecture is structured around modular cloud services:

- Firebase Core: Manages OAuth2 tokens and Cloud Firestore document persistence with rule-level access control.
- Supabase Edge Functions: Deno-based serverless endpoints executing AI RAG inference (`/chat`), eSewa HMAC-SHA256 order signing (`/create-esewa-order`), and OCR verification (`/verify-identity`).
- Supabase Storage: Object storage partitioned into `public-avatars` (5 MB max) and `private-verification-docs` (25 MB max).

3.2 Module Implementation

3.2.1 Authentication and Onboarding Module

The application uses Firebase Authentication for secure user sign-in. Users can register using email and password or Google Sign-In. Upon successful authentication, Firebase issues a secure token which is verified on every database request. Firestore Security Rules enforce strict data access boundaries, ensuring users can only read or modify their own profiles. Figure 3.1 illustrates the onboarding flow.

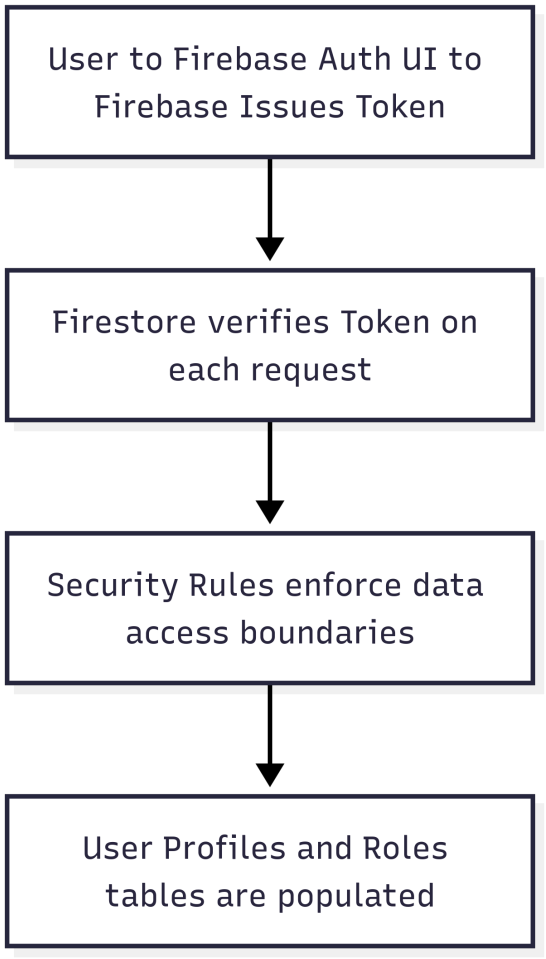


Figure 3.1: Authentication and Onboarding Workflow

3.2.2 AI Tutor Verification Module

To ensure platform safety, tutors must verify their identity before their profiles become public. The tutor uploads photos of their citizenship document and a self-portrait to Supabase Storage. A Supabase Edge Function then uses AI (Groq Vision OCR) to extract the holder's name from the document and compare it against the applicant's profile name, while the

portrait is checked for a real human face. Submissions with a high confidence score are automatically approved, while lower scores are flagged for manual review by an administrator.

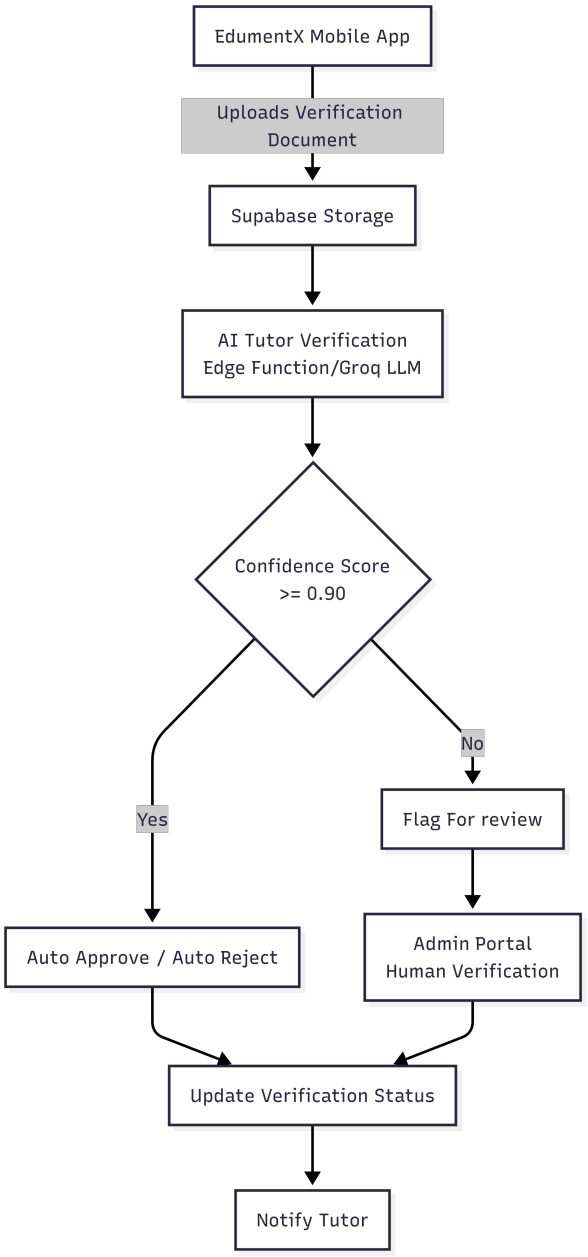


Figure 3.2: Architecture of Tutor Verification Process

3.2.3 Tutor Discovery and Map Module

The application provides an interactive map for students to discover nearby tutors. Using the `expo-location` library, the app fetches the user's current coordinates and displays tutors as custom pins on the map. The Haversine formula is used to calculate the exact distance between the student and the tutor. To ensure smooth performance, the map clusters nearby

pins together when zoomed out.

3.2.4 Enrollment Request Workflow Module

Students can request one-on-one classes by selecting preferred subjects and schedule slots. The request is initially marked as Pending. The tutor can then review the request and choose to accept, decline, or reschedule it. Once accepted, the student is added to the tutor's active roster, and the tutor gains secure access to the student's exact location for home tuition. Figure 3.3 illustrates the lifecycle of a request.

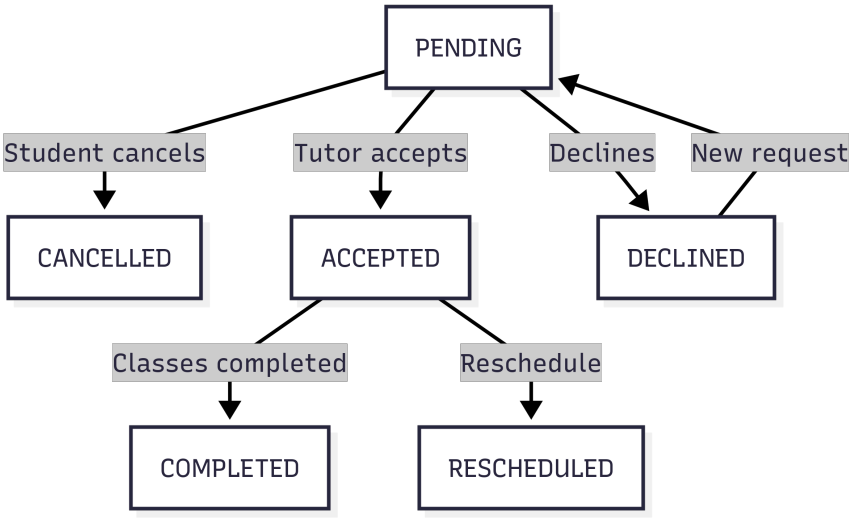


Figure 3.3: Enrollment Request Status Lifecycle

3.2.5 Group Batches Management Module

Tutors have the option to group multiple students into a single batch, ranging from 2 to 6 students. This feature allows tutors to offer shared classes at a reduced monthly rate. Tutors can invite their actively enrolled students into these batches. A visual capacity ring in the UI dynamically shows the remaining available seats in each batch.

3.2.6 In-App Direct Messaging Module

A built-in chat system allows seamless communication between students and tutors without requiring third-party messaging apps. The module generates a unique, deterministic conversation ID for each user pair to maintain privacy. The chat interface supports real-time text messaging, unread message badges, and read receipts to track message status.

3.2.7 Pro Tier and eSewa Payment Module

Tutors can upgrade to a Pro subscription to increase their maximum student capacity. The application integrates with eSewa, Nepal's leading digital wallet, to process these payments.

When a plan is selected, the app generates a pending transaction and redirects the user to the eSewa portal. Upon returning to the app, a backend Edge Function verifies the payment status directly with the eSewa server.

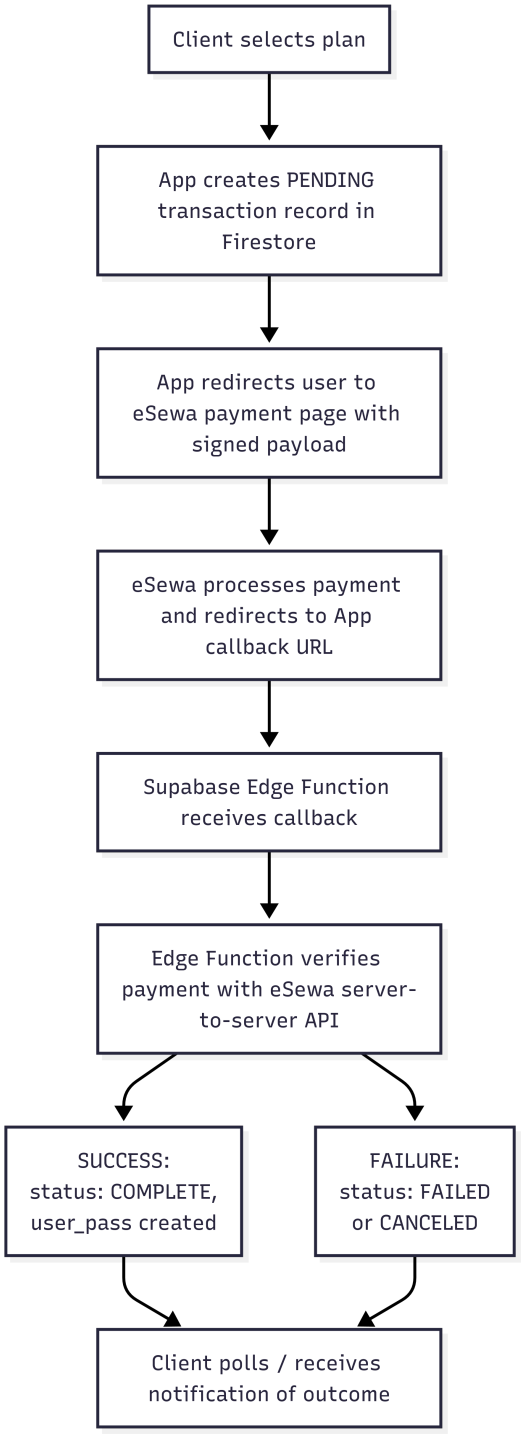


Figure 3.4: eSewa Payment Transaction Flow

3.2.8 Ratings and Reviews Module

To build a reliable community, students can review their tutors after completing their classes. The rating system covers five specific categories: Communication, Subject Knowledge, Punctuality, Clarity, and Engagement. The platform restricts review access only to officially enrolled students, preventing spam and fake ratings. The average rating is automatically recalculated using an atomic database transaction.

3.2.9 AI Chatbot Module

The application includes a Retrieval-Augmented Generation (RAG) chatbot to help students find tutors using natural language. A student can type a prompt like "Find a science tutor in Baneshwor." The AI extracts the intent, searches the Firestore database for matching profiles, and returns a conversational response with direct, clickable links to the recommended tutors.

3.2.10 Admin Dashboard Module

A dedicated administrative portal provides an overview of key platform metrics, such as total registered users and active tutors. The primary function of the admin dashboard is managing the tutor verification queue. Administrators can manually inspect flagged citizenship documents and photos, and officially approve or reject tutor applications.

4. System Design

This chapter presents the requirements, architectural diagrams, data flow diagrams, and database structures of the EdumentX platform.

4.1 Requirement Specification

4.1.1 Functional Requirements

FR1: User Authentication and Onboarding

- Users can register as a Student or Tutor.
- The system supports Google Sign-In and Email/Password authentication.
- Email verification is enforced for email/password signups.

FR2: Tutor Profile Setup and Management

- Tutors can configure teaching subjects, grade levels, and their expected monthly rates.
- Tutors can upload citizenship documents and portrait photos for verification.
- Profiles can be updated anytime to reflect changing availability.

FR3: Location-Based Discovery and Filtering

- Students can discover tutors on a real-time, clustered Google Maps interface.
- The app fetches device location coordinates to center the map.
- Students can filter results by subject, budget, distance, and rating.

FR4: Enrollment and Scheduling

- Students can send enrollment requests specifying subjects and time duration.
- Tutors can accept, decline, or reschedule incoming requests.
- Students can track request status in real-time (Pending, Accepted, Declined).

FR5: Group Batches Creation

- Tutors can create group tuition batches with a capacity of 2 to 6 seats.
- Tutors can assign subjects, schedules, and group pricing to a batch.
- Tutors can directly invite active students to join the batch.

FR6: Real-Time In-App Messaging

- Students and tutors can exchange text messages in real-time.
- The system tracks and displays unread message counts.
- Real-time read receipts allow users to know when messages are seen.

FR7: Ratings and Reviews

- Enrolled students can rate tutors on Communication, Subject Knowledge, Punctuality, Clarity, and Engagement.
- Students can leave written feedback describing their experience.
- The system automatically recalculates and displays the average rating.

FR8: AI Tutor Verification

- The system uses Groq Vision OCR to read uploaded citizenship documents.
- Extracted text is compared against user input, and the face is cross-checked.
- Ambiguous submissions are routed to an admin queue for manual human review.

FR9: Pro Tier Subscription (eSewa Integration)

- Tutors can upgrade to a Pro tier to increase student capacity limits.
- Payments are processed securely via the eSewa digital wallet.
- The backend verifies transaction success directly with the eSewa server.

FR10: Conversational AI Search (RAG)

- Students can input natural language queries (e.g., "Find a math tutor near me under NPR 5000").
- The AI extracts parameters and searches the database automatically.
- The system replies with conversational recommendations and clickable profile cards.

4.1.2 Non-Functional Requirements

- **Performance**: Map rendering and pin clustering must execute smoothly without lagging. AI Chatbot queries should return results within acceptable time limits (under 3 seconds).
- **Security**: Firestore Security Rules must enforce strict data boundaries to prevent unauthorized profile modifications. API requests must include verified Firebase Auth tokens.
- **Privacy**: Exact location data and home addresses of students must remain hidden until an enrollment request is explicitly accepted by the tutor.
- **Reliability**: The mobile application must handle sudden network drops gracefully and display appropriate error messages without crashing.
- **Usability**: The user interface should be clean, intuitive, and navigable with minimal effort for users of all ages.

4.2 System Architecture

The system architecture of EdumentX is built on a scalable, cloud-native three-tier model, designed to separate concerns and ensure maintainability. Figure 4.1 illustrates the high-level interactions between these components.

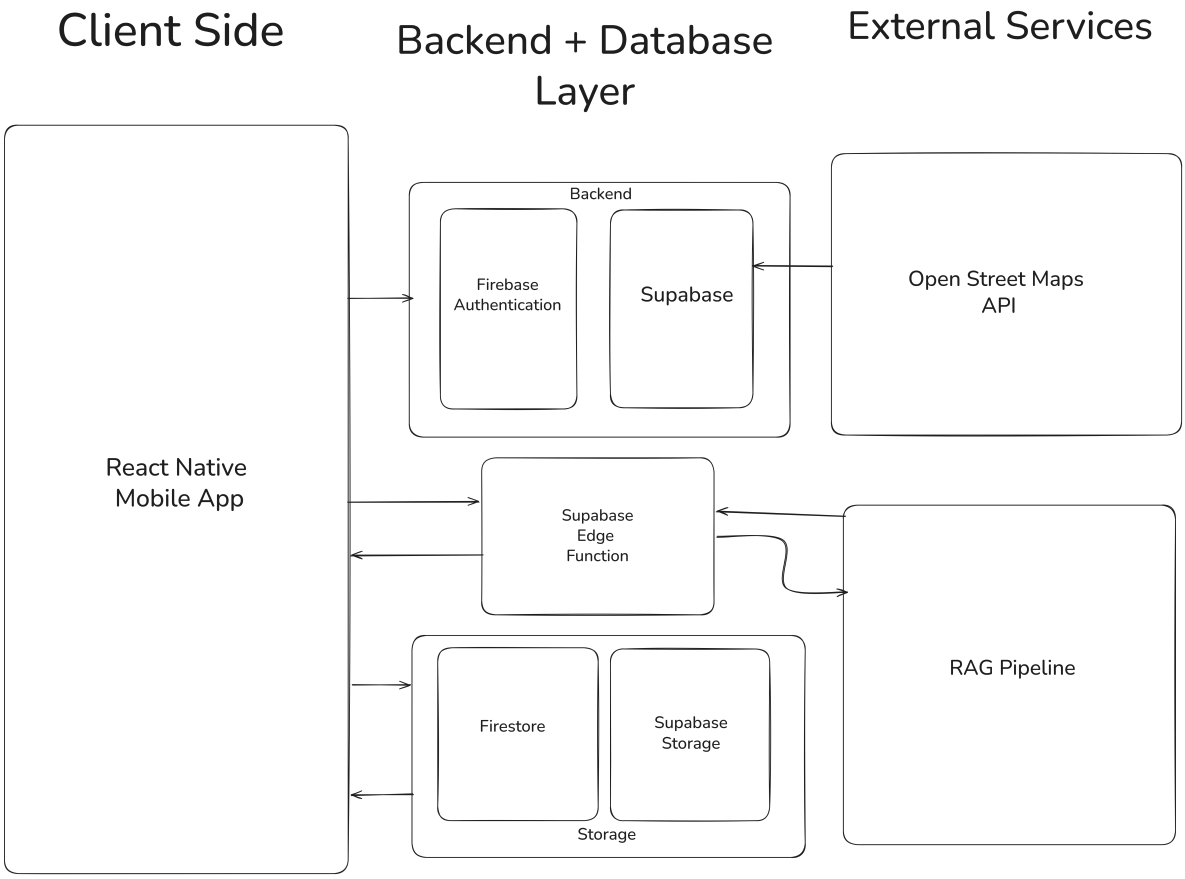


Figure 4.1: System Architecture of EdumentX

The architecture is divided into three primary layers:

1. **Presentation / Client Layer:** Developed using React Native and Expo, this layer provides a cross-platform mobile interface. It utilizes NativeWind for responsive Tailwind CSS styling and integrates native Google Maps for high-performance vector rendering and client-side map clustering.

2. **Application / Serverless Backend Layer:** This middle tier orchestrates business logic using serverless computing. It relies on Firebase Authentication for secure access control and Supabase Edge Functions to handle computationally heavy or secure tasks, such as AI RAG inference, eSewa payment signature generation, and tutor verification processing.

3. **Data Access / Storage Layer:** The foundation of the system is composed of Cloud Firestore for real-time, NoSQL document synchronization, Supabase PostgreSQL with the `pgvector` extension for semantic AI search, and Supabase Storage for securely hosting tutor verification documents and avatars.

4.3 Data Flow Diagrams

4.3.1 Level 0 Context Diagram

The Level 0 Context Diagram establishes the system boundary and identifies how external entities interact with the EdumentX platform. As shown in Figure 4.2, the core system acts as a central hub processing inputs and outputs between Students, Tutors, System Administrators, the eSewa Payment Gateway, and the Groq AI API.

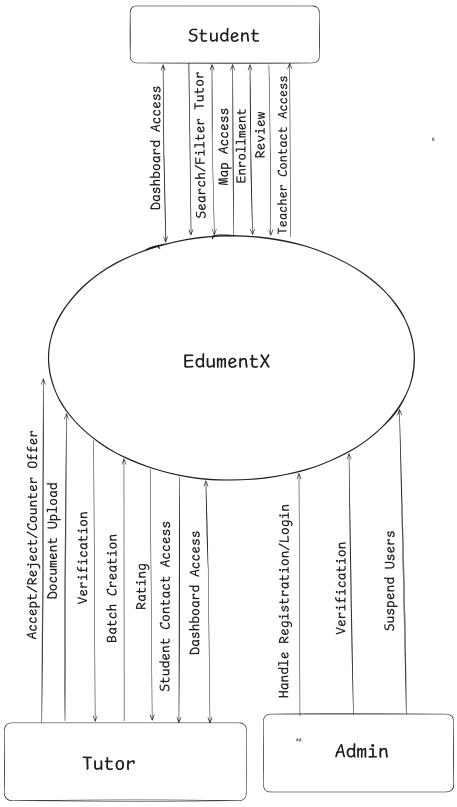


Figure 4.2: Level 0 Data Flow Diagram (Context Diagram)

4.3.2 Level 1 Data Flow Diagram

The Level 1 Data Flow Diagram (Figure 4.3) decomposes the main system into its primary operational modules. It details the specific data exchanges involved in user authentication, map-based tutor discovery, the enrollment request lifecycle, AI chatbot interactions, document verification, and subscription payment processing.

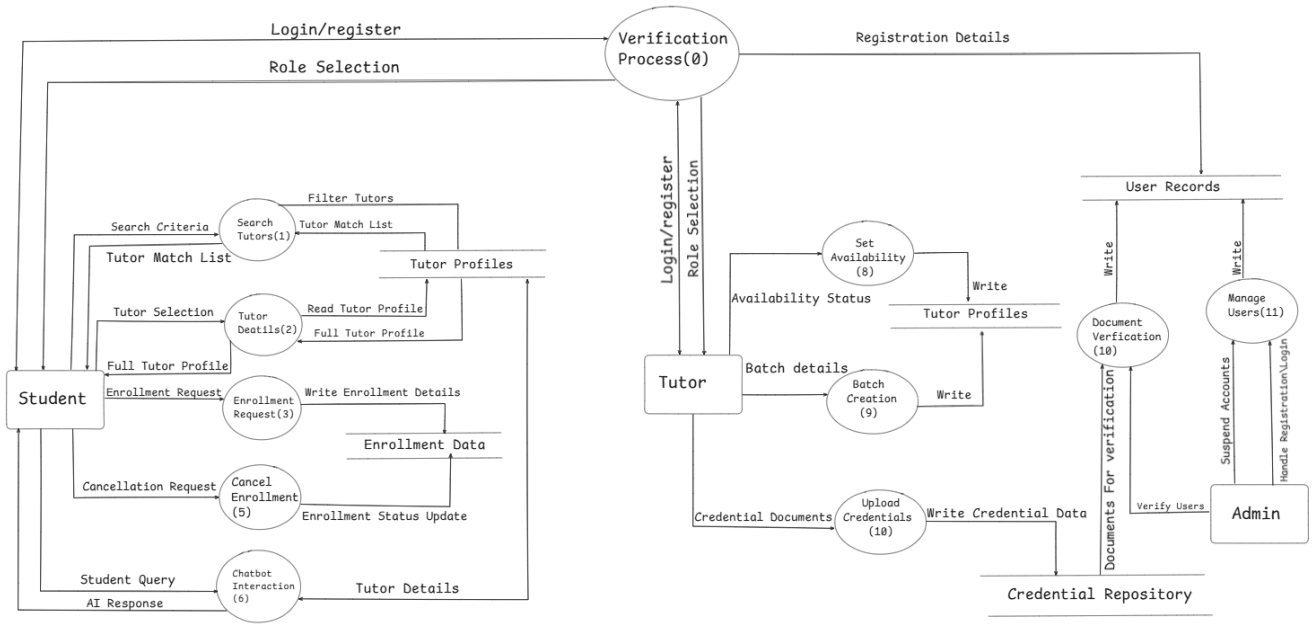


Figure 4.3: Level 1 Data Flow Diagram

4.4 Use Case Diagram

The Use Case Diagram visually represents the functional capabilities available to different actors within the system. Figure 4.4 maps out the specific permissions and actions assigned to Students (e.g., searching, enrolling, chatting), Tutors (e.g., managing profiles, accepting requests, creating batches), and Administrators (e.g., verifying tutor identities and monitoring platform metrics).

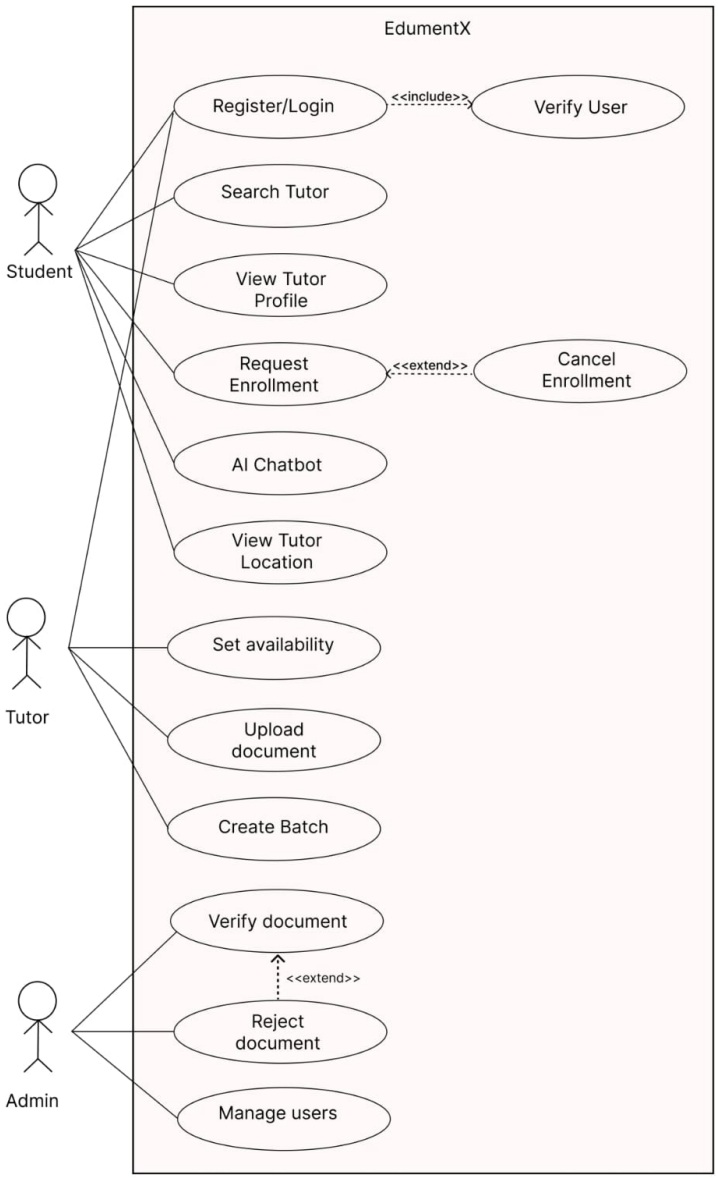


Figure 4.4: Use Case Diagram of EdumentX

4.5 AI RAG Chatbot Workflow

To provide an intuitive search experience, EdumentX implements a Retrieval-Augmented Generation (RAG) pipeline. Figure 4.5 illustrates how the system processes natural language queries to deliver accurate, context-aware tutor recommendations.

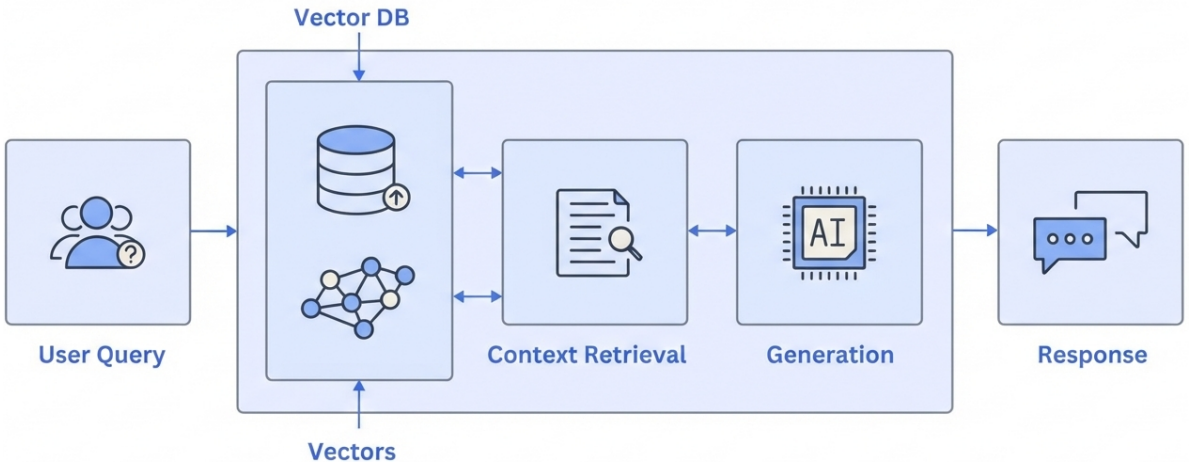


Figure 4.5: RAG-Based AI Chatbot Architecture

The chatbot processes queries through the following pipeline:

1. **Query Input:** The student submits a natural language question (e.g., "Need a physics tutor in Baneshwor") alongside their recent chat history for context.

2. **Parameter Extraction:** A specialized Groq LLM prompt extracts key search parameters such as subject, grade level, maximum budget, and preferred location.

3. **Database Retrieval:** The extracted parameters are used to query Cloud Firestore and the pgvector database to fetch highly relevant candidate tutor profiles.

4. **Prompt Construction:** The retrieved tutor details are injected into a final system prompt, grounding the AI's knowledge in real, current database records.

5. **Response Generation:** The model synthesizes a conversational, personalized reply recommending the best-matching tutors based on the student's constraints.

6. **Card Rendering:** The mobile application parses the response and renders interactive UI cards, allowing the student to click directly through to a tutor's full profile.

4.6 Database Schema

Table 4.1 summarizes the primary Firestore collections.

Table 4.1: Primary Firestore Document Collections and Schema

Collection Path	**Key Fields**	**Description**
users/{uid}	uid, email, role, status	Main user account record
users/{uid}/ studentProfile/default	fullName, phone, grade, subjects[]	Student profile subdocument
users/{uid}/ tutorProfile/default	fullName, headline, monthlyRateNpr, rating	Canonical tutor profile subdocument
users/{uid}/ locationAccess/ {tutorUid}	studentUid, tutorUid, enrollmentId, status	Active enrollment location marker
tutors/{uid}	uid, fullName, monthlyRateNpr, isAvailable	Public search mirror
enrollmentRequests/ {tutorUid}/requests/{id}	tutorUid, studentUid, subjects[], status	Student enrollment request
enrollments/{tutorUid}/ roster/{id}	enrollmentId, tutorUid, studentUid, slotKey	Active student roster row
batches/{tutorUid}/ classes/{batchId}	batchId, tutorUid, name, subject	Group tuition batch
conversations/{id}	participantA, participantB, lastMessage	1:1 chat conversation thread
reviews/{tutorUid}/ reviews/{id}	tutorUid, studentUid, score, categoryRatings	Verified student review

5. Results and Discussion

This chapter describes the development approach, test environment, test cases, and demonstration of the completed mobile application.

5.1 Software Development Approach

The development followed an iterative approach over six months. Work was divided into short development cycles, where each feature was implemented and tested on real Android devices before moving forward.

5.2 Testing, Evaluation and Validation

5.2.1 Test Environment Specification

Table 5.1 lists the environment used for running automated tests and validating the application.

Table 5.1: Test Environment Specifications

Component	**Environment**	**Version**
Development OS	Linux x86_64	Node.js v20.19.4, npm 10.8.2
Mobile Framework	React Native / Expo	React Native 0.81.5, Expo SDK 54
Test Phones	Realme 10, OnePlus Nord CE5	Android 13 / 14
Database Emulator	Firebase Firestore Emulator	firebase-tools v15.27.0
AI Models	Groq Cloud API (chatbot RAG / vision OCR)	LLaMA 3.1 70B / Qwen 3.6 27B

5.2.2 Unit Test Cases

Table 5.2: Unit Test Case UT-01 – Asia/Kathmandu Timezone Conversion

Field	**Details**
Test ID / Name	UT-01: Timezone Normalization and Date Helper
Target Module	`src/services/enrollments/derived.ts` (`todayIsoInKtm`)
Preconditions	Clock set to UTC timestamp
Input Test Data	`2026-08-23T19:30:00.000Z`
Expected Result	Date converts to Nepal date `2026-08-24`
Actual Result	Returns `2026-08-24`, weekday `"mon"`
Status	PASS

Table 5.3: Unit Test Case UT-02 – SlotKey Syntax Validation

Field	**Details**
Test ID / Name	UT-02: SlotKey Syntax Parser and Guard
Target Module	`src/services/enrollments/derived.ts` (`parseSlotKey`)
Preconditions	Standard day and slot keys defined
Input Test Data	Input A: `"mon:5-7"`; Input B: `"mon-5-7"` (bad delimiter)
Expected Result	Input A parses correctly; Input B returns `null`
Actual Result	Input A parsed as { `day: "mon", slot: "5-7"` }; Input B returns `null`
Status	PASS

Table 5.4: Unit Test Case UT-03 – Haversine Great-Circle Distance Calculation

Field	**Details**
Test ID / Name	UT-03: Haversine Geographic Distance
Target Module	`src/lib/location/distance.ts` (`haversineKm`)
Preconditions	WGS84 coordinates of two points in Kathmandu Valley
Input Test Data	Point 1 (Talchhikhel: 27.6588, 85.3247); Point 2 (Ratna Park: 27.7058, 85.3149)
Expected Result	Distance is approximately 5.32 km
Actual Result	Computed distance equals 5.316 km
Status	PASS

Table 5.5: Unit Test Case UT-04 – Dynamic Today's Sessions Derivation

Field	**Details**
Test ID / Name	UT-04: Today's Sessions Derivation
Target Module	`src/services/enrollments/derived.ts` (`deriveTodaySessions`)
Preconditions	Roster containing mixed active and expired rows
Input Test Data	Today: Monday; Row 1 (`mon:5-7`, active); Row 2 (`tue:5-7`, active)
Expected Result	Only Row 1 is returned for today's schedule
Actual Result	Filtered list returns Row 1 only
Status	PASS

Table 5.6: Unit Test Case UT-05 – SeatsRing SVG Capacity Math

Field	**Details**
Test ID / Name	UT-05: Group Batch SeatsRing Math
Target Module	`src/components/domain/seatsRingMath.ts` (`seatsRingFrac`)
Preconditions	Max seats: 6, Enrolled: 4
Input Test Data	Max: 6, Enrolled: 4
Expected Result	Available fraction is $2/6 = 0.3333$
Actual Result	Fraction equals 0.3333
Status	PASS

5.2.3 Integration Test Cases

Table 5.7: Integration Test Case IT-01 – Signup, Email Verification and Role Routing

Field	**Details**
Test ID / Name	IT-01: Authentication to Role Dashboard Routing
Target Modules	`EmailSignUp.tsx`, `_layout.tsx`
Preconditions	User registered with verified email
Procedure	1. Log in. 2. Root layout reads role. 3. Navigate.
Expected Result	Student user is routed to `/student-home`
Actual Result	Router successfully opens `/student-home`
Status	PASS

Table 5.8: Integration Test Case IT-02 – Tutor Verification Upload and AI Screening

Field	**Details**
Test ID / Name	IT-02: Tutor Document Upload and OCR Check
Target Modules	`TutorProfileScreen.tsx`, Supabase `/verify-identity`
Preconditions	Citizenship image uploaded to storage
Procedure	1. Submit tutor profile. 2. Edge Function runs OCR check.
Expected Result	Extracted name matches profile name; portrait photo accepted
Actual Result	Exact name match approved automatically with confidence score 1.0; a deliberate misspelling (one extra letter) scored below the match threshold and was correctly routed to the manual review queue
Status	PASS

Table 5.9: Integration Test Case IT-03 – Map Pin Clustering and Selection

Field	**Details**
Test ID / Name	IT-03: Native Map Clustering
Target Modules	`TutorMap.tsx`, `useTutorClustering.ts`
Preconditions	Tutors with valid coordinates in Kathmandu
Procedure	1. Move map view. 2. Supercluster groups points. 3. Tap a pin.
Expected Result	Pins cluster properly; tapping single pin opens preview sheet
Actual Result	Preview sheet opens showing tutor rate and distance
Status	PASS

Table 5.10: Integration Test Case IT-04 – Enrollment Request to Roster Transaction

Field	**Details**
Test ID / Name	IT-04: Atomic Enrollment Acceptance
Target Modules	`TutorInbox.tsx`, `FirebaseEnrollmentRepository.ts`
Preconditions	Pending request exists; tutor has available capacity
Procedure	1. Tap Accept. 2. Choose slot `"mon:5-7"`. 3. Save.
Expected Result	Roster row created, capacity count updated in transaction
Actual Result	Transaction succeeds; request disappears from inbox
Status	PASS

Table 5.11: Integration Test Case IT-05 – eSewa Payment and Pro Activation

Field	**Details**
Test ID / Name	IT-05: eSewa Payment Verification
Target Modules	`ProUpgradeScreen.tsx`, Supabase `/create-esewa-order`
Preconditions	Tutor selects Monthly Pro plan
Procedure	1. Submit payment in WebView. 2. Verify callback HMAC signature.
Expected Result	Server verifies HMAC and updates user tier to Pro
Actual Result	Subscription status updates to Pro immediately
Status	PASS

5.2.4 System Journey Test Cases

Table 5.12: System Journey Test Case SYS-J1 – Student Discovery to Enrollment

Field	**Details**
Test ID / Name	SYS-J1: Student Booking Journey
Actor	Student User
Journey Steps	1. Open Map Search → Filter by Grade 10 Math → Tap Pin. 2. View tutor profile → Check details and ratings. 3. Tap Enroll → Pick 3-Month plan with slot `"tue:5-7"` → Submit request.
Expected Result	Request appears in Student Enrollment tab as Pending
Actual Result	Request saved to Firestore; tutor inbox updates in real time
Status	PASS

Table 5.13: System Journey Test Case SYS-J2 – Tutor Onboarding and Schedule Setup

Field	**Details**
Test ID / Name	SYS-J2: Tutor Verification and Schedule Setup
Actor	Tutor User and Admin
Journey Steps	1. Fill profile and upload citizenship → Routed to pending screen. 2. Admin opens Verification Queue → Reviews document → Approves. 3. Tutor opens Capacity Screen → Sets available weekly slots.
Expected Result	Tutor profile becomes active and visible on map search
Actual Result	Profile status updates and appears in discovery queries
Status	PASS

Table 5.14: System Journey Test Case SYS-J3 – Pro Subscription Upgrade

Field	**Details**
Test ID / Name	SYS-J3: Pro Plan Upgrade via eSewa
Actor	Tutor User
Journey Steps	1. Tap Pro Upgrade banner on dashboard. 2. Choose plan → Enter eSewa test credentials → Confirm. 3. Screen redirects back to app → Success message shown.
Expected Result	Student capacity limit increases from 5 to 10+
Actual Result	Pro tier active; new capacity limits applied
Status	PASS

5.2.5 AI Identity Verification and Vision OCR Evaluation

The verification pipeline was exercised against real citizenship certificates and profile photographs submitted through the live sandbox environment during onboarding testing rather than against a synthetic benchmark dataset. Each submission produces a verdict recorded in the function logs and mirrored into the applicant's Firestore profile document, so every figure below is traceable to a stored verdict. Table 5.15 summarizes the observed behaviour.

Table 5.15: AI Identity Verification – Observed Sandbox Behaviour

Scenario	**Observed Outcome**
Citizenship card with English name matching the profile name	OCR extracted the full name exactly; similarity score 1.0; face detected (score 0.9); auto-approved
Citizenship card with a one-letter spelling difference from the profile name	Similarity fell below the acceptance threshold; routed to the admin manual-review queue instead of being rejected outright
Profile photo containing a clear human portrait	Face detected with score 0.9
Model output quality control	Chain-of-thought leakage from the vision model was detected in early runs and eliminated by disabling reasoning tokens at the API level
Failure handling	When the upstream AI provider is unreachable, the function returns a `manual_review` verdict instead of crashing, so no applicant is lost

5.2.6 Security Test Cases

Table 5.16: Security Test Case ST-01 – Student Location Privacy Protection

Field	**Details**
Test ID / Name	ST-01: Unauthorized Student Location Read
Security Rule	`match /users/{userId}/studentProfile/default`
Scenario	A tutor without an active enrollment tries to read a student's full location.
Expected Result	Firestore blocks the read with permission denied
Actual Result	Read blocked; unlocked only when active enrollment record exists
Status	PASS

Table 5.17: Security Test Case ST-02 – Cross-User Profile Modification Guard

Field	**Details**
Test ID / Name	ST-02: Cross-User Profile Write Block
Security Rule	`match /users/{userId}`
Scenario	User A tries to edit User B's profile document.
Expected Result	Firestore blocks the write with permission denied
Actual Result	Unauthorized write blocked
Status	PASS

Table 5.18: Security Test Case ST-03 – Free Tier Capacity Cap Enforcement

Field	**Details**
Test ID / Name	ST-03: Free Tier Capacity Cap
Security Rule	`match /enrollments/{tutorUid}/roster/{id}`
Scenario	A free-tier tutor with 5 active students tries to add a 6th student.
Expected Result	Security rules block addition when count reaches 5
Actual Result	Addition blocked until tutor upgrades to Pro
Status	PASS

Table 5.19: Security Test Case ST-04 – Chat Thread Access Authorization

Field	**Details**
Test ID / Name	ST-04: Chat Thread Access Check
Security Rule	`match /conversations/{id}`
Scenario	An unrelated User C tries to read messages in User A and B's chat.
Expected Result	Firestore rules verify user is one of the participants
Actual Result	Read blocked for non-participants
Status	PASS

5.2.7 Test Traceability Summary

Table 5.20 maps each functional requirement to its verified test cases.

Table 5.20: Test Suite Traceability Matrix

Requirement	**Description**	**Verified Test Case IDs**
FR1	User Authentication and Onboarding	IT-01, ST-02
FR2	Tutor Profile Setup and Management	IT-02, SYS-J2
FR3	Location-Based Discovery and Filtering	UT-03, IT-03, SYS-J1
FR4	Enrollment and Scheduling	UT-01, UT-02, UT-04, IT-04, SYS-J1
FR5	Group Batches Creation	UT-05, SYS-J2
FR6	Real-Time In-App Messaging	ST-04
FR7	Ratings and Reviews	ST-02
FR8	AI Tutor Verification	IT-02, SYS-J2, Table 5.15
FR9	Pro Tier Subscription (eSewa)	IT-05, SYS-J3, ST-03
FR10	Conversational AI Search (RAG)	Manual demonstration (live queries)
NFR1–NFR5	Performance, Privacy, Security	UT-01–05, ST-01–04

6. Conclusions

This project set out to solve a problem we had all experienced in our own neighbourhoods: finding a dependable home tutor in the Kathmandu Valley still depends on word-of-mouth, pamphlets, and unverified social media posts. Over the minor project we built EdumentX, an Android application that replaces that guesswork with a searchable, verified marketplace connecting students directly with tutors. All of the objectives set out in our proposal were met, and some grew beyond what we first planned after the mid-term review.

Our circumstances shaped the technical decisions as much as our preferences did. With no institutional funding or international payment method, we had to build on free tiers, and that constraint became one of the most useful lessons of the project. Firebase Authentication and Cloud Firestore handle identity and the primary database; their real-time listeners drive the live tutor dashboards and enrollment updates without any custom server. Object storage, serverless business logic and vector search for the chatbot run on Supabase. The platform costs nothing per month to operate at demo scale, and holding that line forced us to understand each service instead of reaching for paid shortcuts.

The features themselves came together into a coherent product. Students find tutors on an interactive Google Maps view where pins are clustered for performance and ranked by real Haversine distance from their location. The retrieval-augmented chatbot answers plain English questions from live database records rather than invented suggestions. Enrollment requests pass through an atomic transaction that respects capacity, group batches let one tutor teach many students together, and messaging keeps both sides coordinated inside the app. The eSewa integration, completed in the sandbox, showed that a local digital wallet can drive a subscription end to end.

The part we are most satisfied with is tutor verification. A vision model reads the uploaded citizenship certificate and compares the holder's name with the profile, while the photograph is checked for a real human face. Clear passes are approved automatically; anything doubtful goes to the admin queue with an AI pre-screen verdict attached. Building this taught us how fragile AI features are in practice: a genuine certificate with one misspelled letter fell below our match threshold, and the vision model leaked its internal reasoning into outputs until we constrained it at the API level. Finally, we verified our own work seriously: sixty-four unit tests cover the scheduling, distance and capacity logic, twenty-one lint tests keep the design tokens consistent, and seven Firestore rule suites confirm that location privacy and

chat access behave as specified. Together with manual testing on physical devices, this gives us confidence that EdumentX is a working platform, not merely a demonstration.

7. Limitations and Future Enhancement

7.1 Limitations

- iOS Platform Verification: The native iOS project has been generated from the shared codebase and compiles, but systematic testing on physical iPhone devices has not been carried out; all functional validation was performed on Android. Map rendering on iOS would also fall back to Apple Maps, as Google Maps credentials are currently provisioned for Android only.
- eSewa Sandbox: The payment integration runs in the eSewa test environment (`EPAYTEST`). Production deployment requires a registered business merchant account.
- Service Coverage: Tutor discovery content is concentrated in the Kathmandu Valley, where seed data exists, and geocoding relies on the keyless Nominatim service, which enforces a strict fair-use rate limit of roughly one request per second.
- In-Person Focus: The platform is designed specifically for physical home tutoring; online video classes are not included.
- Push Notification Delivery: Background message notifications are wired through a Cloud Function trigger and Expo push tokens; delivery behaviour still varies across device vendors because of aggressive battery optimization on some Android phones.

7.2 Future Enhancement

- iOS Release: Complete on-device quality assurance of the existing iOS build, provision Google Maps for iOS, and distribute the app to testers through TestFlight.
- Production Payments: Integrate live eSewa and Khalti merchant payment gateways with automated escrow payout.
- Online Tutoring Mode: Extend session delivery beyond physical home visits by adding integrated video calls and digital learning materials.

- Nepali Language Support: Add Nepali language translation for the user interface and AI chatbot replies.
- Tutor Analytics: Provide tutors with dashboard charts showing profile views, search appearances, and subject demand trends in their area.

References

[1] M. Bray. *The Shadow Education System: Private Tutoring and Its Implications for Planners*. Fundamentals of educational planning. UNESCO, International Institute for Educational Planning, Paris, 2007.

[2] M. Bray and C. Lykins. *Shadow Education: Private Supplementary Tutoring and Its Implications for Policy Makers in Asia*. CERC monograph series in comparative and international education and development. Asian Development Bank, Manila, 2012.

[3] Y. C. Kim and J. H. Jung. Shadow education as worldwide curriculum studies. *Curriculum Studies Worldwide*, 1(1):1–15, 2019.

[4] Amrit Thapa. Does private school competition improve public school performance? the case of nepal. *International Journal of Educational Development*, 33(4):358–366, 2013.

[5] Central Bureau of Statistics. National population and housing census 2021: National report, 2023.

[6] Sheng-Hung Chung and Seng Chee Tan. Mentor – intelligent mobile online peer tutoring application for face-to-face and remote peer tutoring. *ASCILITE Publications*, pages 386–391, 2022.

[7] R. K. Sharma, A. Patel, and M. Singh. Local tutor finder application using flutter. *Journal of Emerging Technologies and Innovative Research (JETIR)*, 13(2):267–275, 2026.

[8] N. Shari, F. Ahmad, and R. Hassan. iteachu: Ai-driven tutor matching with geolocation integration. *International Journal of Research in Science and Innovation (IJRSI)*, 12(3):145–158, 2025.

[9] Jean-Charles Rochet and Jean Tirole. Platform competition in two-sided markets. *Journal of the European Economic Association*, 1(4):990–1029, 2003.

[10] Roger W. Sinnott. Virtues of the haversine. *Sky and Telescope*, 68(2):158, 1984.

[11] Yury Shevchenko and Ulf-Dietrich Reips. Geofencing in location-based behavioral research: Methodology, challenges, and implementation. *Behavior Research Methods*, 56, 2023.

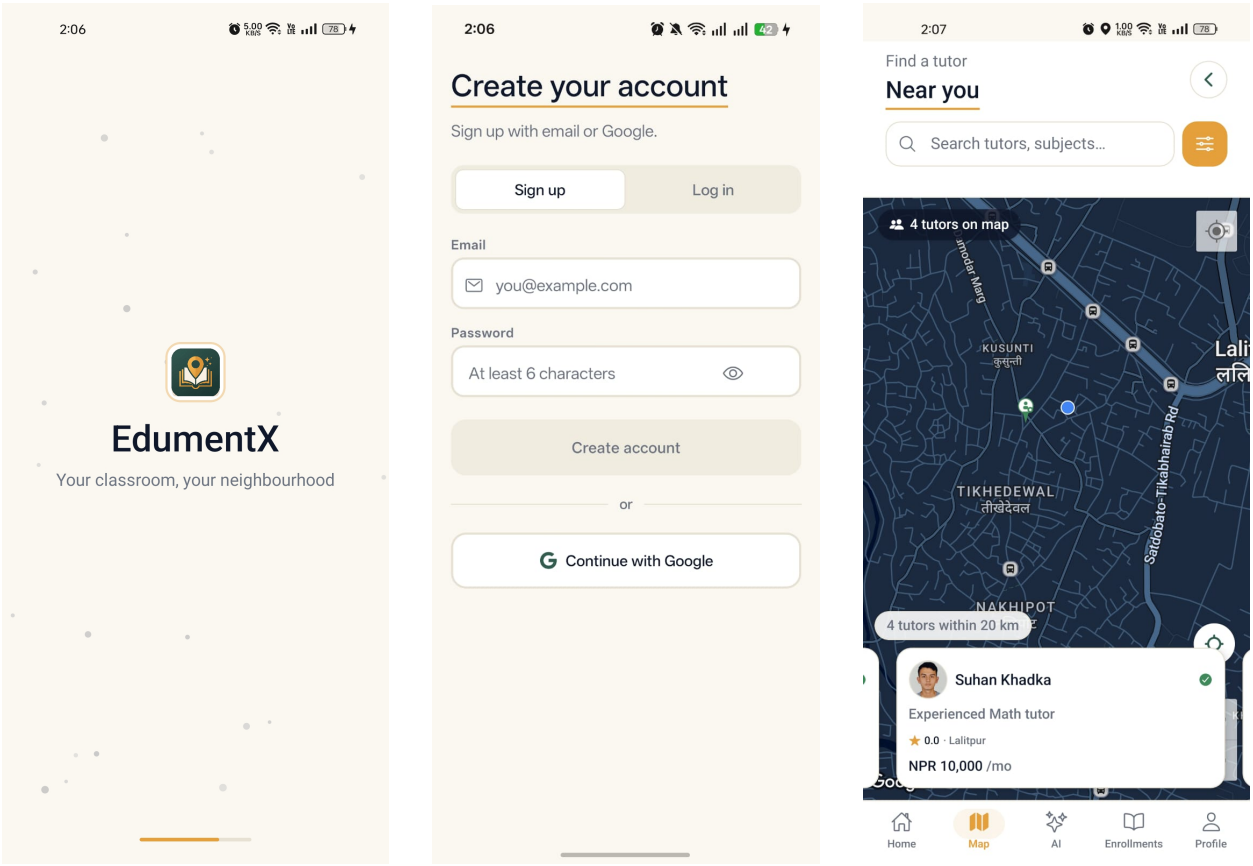
[12] Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K"uttler, Mike Lewis, Wen-tau Yih, Tim Rockt"aschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

[14] Jakub Swacha and Michał Gracel. Retrieval-augmented generation (rag) chatbots for education: A survey of applications. *Applied Sciences*, 15(8):4234, 2025.

Appendices

Appendix: Application Screenshots



(a) Onboarding **(b) Login & Sign-In** **(c) Student Map View**

Figure 7.1: EdumentX Mobile Application Screenshots: (a) Onboarding, (b) Authentication, and (c) Student Map Discovery.